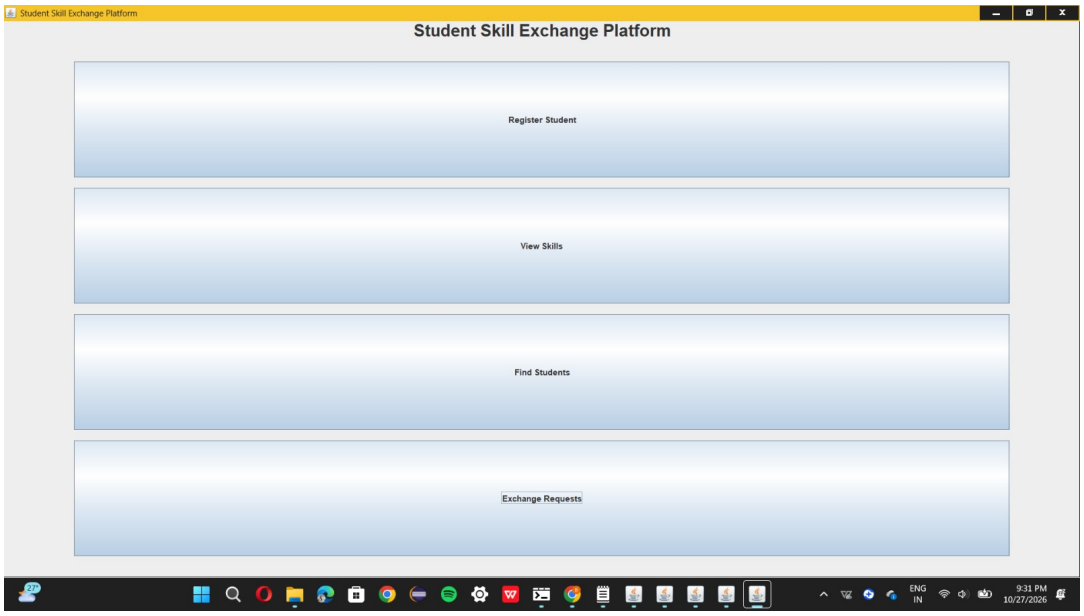
STUDENT SKILL MANAGEMENT SYSTEM

A Java Swing Application for Registering Students, Sharing Skills and Managing Skill-Exchange Requests



Home screen of the Student Skill Exchange Platform

Technology	Java (Swing / AWT)
Data Storage	Text files (data/students.txt, data/requests.txt)
Source File	StudentSkillExchange.java

1. Introduction

The **Student Skill Management System** (Student Skill Exchange Platform) is a desktop application that helps students share what they know and learn what they need. Every student registers with the skills they can *teach* and the skills they want to *learn*. Other students can then be found by name, department or skill, and a skill-exchange request can be sent – for example, one student offers SQL in exchange for learning Java.

The application is built with Java Swing and keeps all of its data in simple text files, so it runs without a database or any extra installation.

2. Objectives

- Provide an easy way to register students along with their skills to teach and to learn.
- Display all registered students and their skills in a clear table.
- Allow quick searching by student name, department or skill.
- Let students create and track skill-exchange requests with a status such as *Pending*.
- Store data permanently in files so records are available the next time the program is opened.

3. Modules of the System

Module	Description
Register Student	Collects name, email, department, year, skills to teach and learn, validates them and saves them to data/students.txt.
View Skills	Reads students.txt and shows Name, Department, Teach and Learn in a table.
Find Students	Searches every field of the records for the keyword and shows matching students.
Exchange Requests	Creates a new request or shows all saved requests (From, To, Offered, Wanted, Status).

4. Tools and Technologies

Item	Details
Language	Java (JDK 8 or above)
GUI Library	javax.swing and java.awt (JFrame, JPanel, JButton, JOptionPane, JTable, DefaultTableModel)
File Handling	java.io (BufferedReader, FileReader, FileWriter, PrintWriter)
Data Storage	Plain text files in a data folder, using the pipe (|) character as a field separator
IDE	Eclipse / any Java IDE

5. Data Storage Design

Each student is stored as one line in **data/students.txt**. The six fields are separated by the pipe character (|):

```
name | email | department | year | skill_to_teach | skill_to_learn
```

The screenshot below shows the students.txt file after two students were registered through the application.

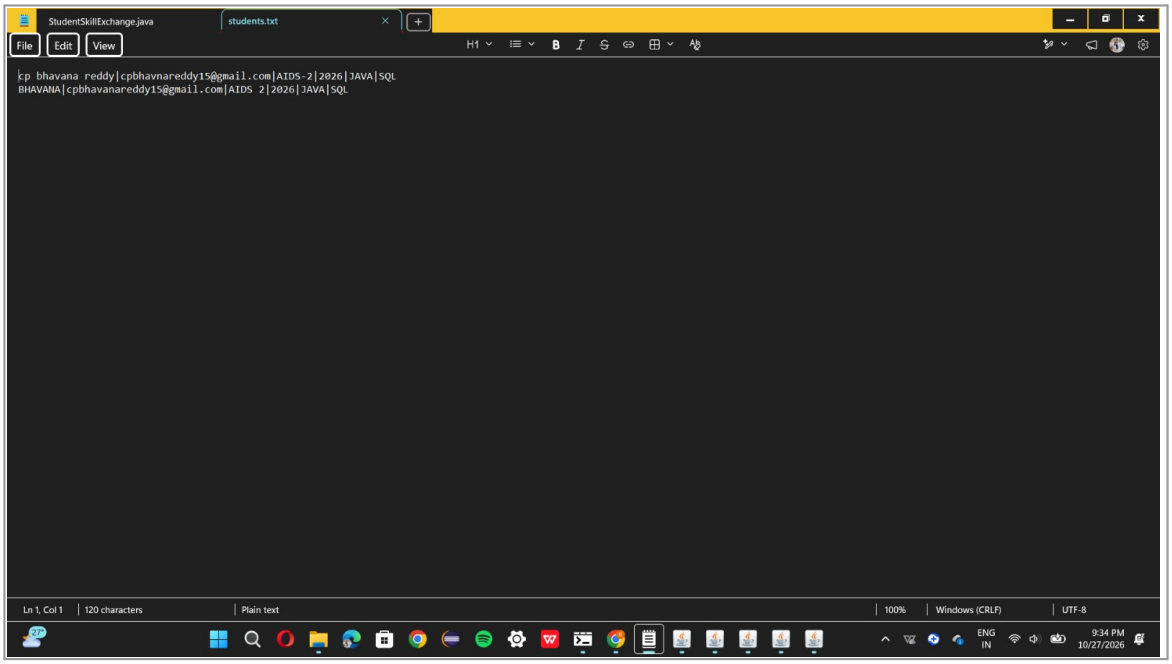


Figure 1: Contents of students.txt (one student per line)

Exchange requests are stored in **data/requests.txt** in the same way, using five fields:

```
from | to | offered_skill | wanted_skill | status

Example:
ananya|bhavana|SQL|JAVA|Pending
```

Sample content of students.txt:

```
cp bhavana reddy|cpbhavnareddy15@gmail.com|AIDS-2|2026|JAVA|SQL
BHAVANA|cpbhavanareddy15@gmail.com|AIDS 2|2026|JAVA|SQL
```

6. Implementation – Development Stages

The project was developed in five stages. Every stage is a complete, runnable version of the single class **StudentSkillExchange.java**; each new stage adds one module to the previous one. The full source code of every stage is given below, together with the output screens it produces.

Stage	What was added
Stage 1	Main window with four buttons (placeholder messages)
Stage 2	Register Student – form, validation and saving to students.txt
Stage 3	View Skills – table of all registered students
Stage 4	Find Students – keyword search; reading code moved to loadStudents()
Stage 5	Exchange Requests, stronger validation, shared helper methods (final version)

Stage 1: Main Window

The first version builds only the user interface. The class extends `JFrame` (700 × 450, centred on screen). A heading label is placed at the top and four buttons are arranged in a 4 × 1 `GridLayout`. Each button just shows a message saying that the feature will be added next.

```
import javax.swing.*;
import java.awt.*;

public class StudentSkillExchange extends JFrame {

    public StudentSkillExchange() {
        setTitle("Student Skill Exchange Platform");
        setSize(700, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JLabel title = new JLabel(
            "Student Skill Exchange Platform",
            SwingConstants.CENTER
        );

        title.setFont(new Font("Arial", Font.BOLD, 24));

        JButton registerButton = new JButton("Register Student");
        JButton skillsButton = new JButton("View Skills");
        JButton searchButton = new JButton("Find Students");
        JButton requestButton = new JButton("Exchange Requests");

        JPanel panel = new JPanel(new GridLayout(4, 1, 15, 15));
        panel.setBorder(BorderFactory.createEmptyBorder(30, 100, 30, 100));

        panel.add(registerButton);
        panel.add(skillsButton);
        panel.add(searchButton);
        panel.add(requestButton);

        add(title, BorderLayout.NORTH);
        add(panel, BorderLayout.CENTER);

        registerButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Student registration will be added next!")
        );

        skillsButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Skill management will be added next!")
        );
```

```
        searchButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Student search will be added next!")
        );

        requestButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Exchange requests will be added next!")
        );
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new StudentSkillExchange().setVisible(true)
        );
    }
}
```

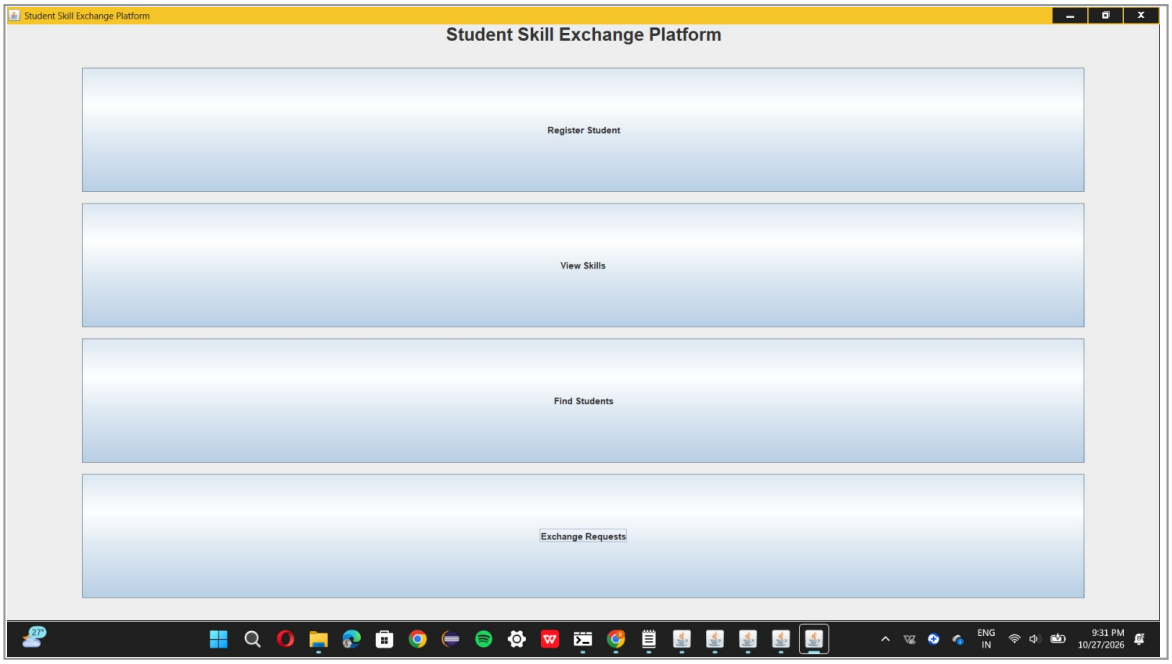


Figure 2: Main window with Register Student, View Skills, Find Students and Exchange Requests

Stage 2: Register Student

The Register Student button now calls **registerStudent()**. A `JOptionPane` confirm dialog with six text fields collects the student details. All fields are checked for emptiness; the record is then appended to **data/students.txt** as one pipe-separated line (the *data* folder is created if it does not exist) and a confirmation message is shown.

```
import javax.swing.*;
import java.awt.*;
import java.io.*;

public class StudentSkillExchange extends JFrame {

    private final String DATA_FILE = "data/students.txt";

    public StudentSkillExchange() {
        setTitle("Student Skill Exchange Platform");
        setSize(700, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JLabel title = new JLabel(
            "Student Skill Exchange Platform",
            SwingConstants.CENTER
        );
```

```
        title.setFont(new Font("Arial", Font.BOLD, 24));

        JButton registerButton = new JButton("Register Student");
        JButton skillsButton = new JButton("View Skills");
        JButton searchButton = new JButton("Find Students");
        JButton requestButton = new JButton("Exchange Requests");

        JPanel panel = new JPanel(new GridLayout(4, 1, 15, 15));
        panel.setBorder(
            BorderFactory.createEmptyBorder(30, 100, 30, 100)
        );

        panel.add(registerButton);
        panel.add(skillsButton);
        panel.add(searchButton);
        panel.add(requestButton);

        add(title, BorderLayout.NORTH);
        add(panel, BorderLayout.CENTER);

        registerButton.addActionListener(e -> registerStudent());

        skillsButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Skill management is coming next.")
        );

        searchButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Student search is coming next.")
        );

        requestButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Exchange requests are coming next.")
        );
    }

    private void registerStudent() {
        JTextField name = new JTextField();
        JTextField email = new JTextField();
        JTextField department = new JTextField();
        JTextField year = new JTextField();
        JTextField teach = new JTextField();
        JTextField learn = new JTextField();

        Object[] fields = {
            "Student Name:", name,
            "Email:", email,
            "Department:", department,
            "Year of Study:", year,
            "Skills to Teach:", teach,
            "Skills to Learn:", learn
        };

        int result = JOptionPane.showConfirmDialog(
            this, fields, "Student Registration",
            JOptionPane.OK_CANCEL_OPTION
        );

        if (result != JOptionPane.OK_OPTION) {
            return;
        }

        if (name.getText().trim().isEmpty()
                || email.getText().trim().isEmpty()
                || department.getText().trim().isEmpty()
                || year.getText().trim().isEmpty()
                || teach.getText().trim().isEmpty()
                || learn.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Please fill in all fields.");
            return;
        }
```

```
        try {
            File folder = new File("data");
            if (!folder.exists()) {
                folder.mkdirs();
            }

            try (PrintWriter out = new PrintWriter(
                    new FileWriter(DATA_FILE, true))) {
                out.println(name.getText().trim() + "|"
                    + email.getText().trim() + "|"
                    + department.getText().trim() + "|"
                    + year.getText().trim() + "|"
                    + teach.getText().trim() + "|"
                    + learn.getText().trim());
            }

            JOptionPane.showMessageDialog(this,
                "Student registered successfully!");

        } catch (IOException ex) {
            JOptionPane.showMessageDialog(this,
                "Could not save student: " + ex.getMessage());
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new StudentSkillExchange().setVisible(true)
        );
    }
}
```

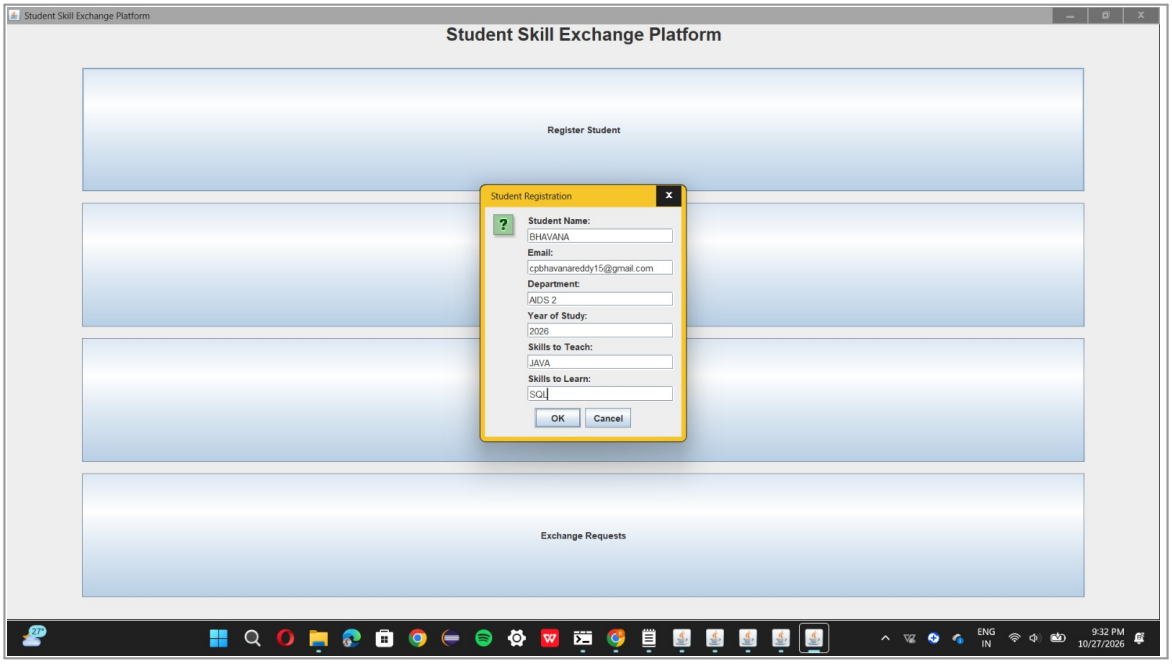


Figure 3: Student Registration form filled with sample data

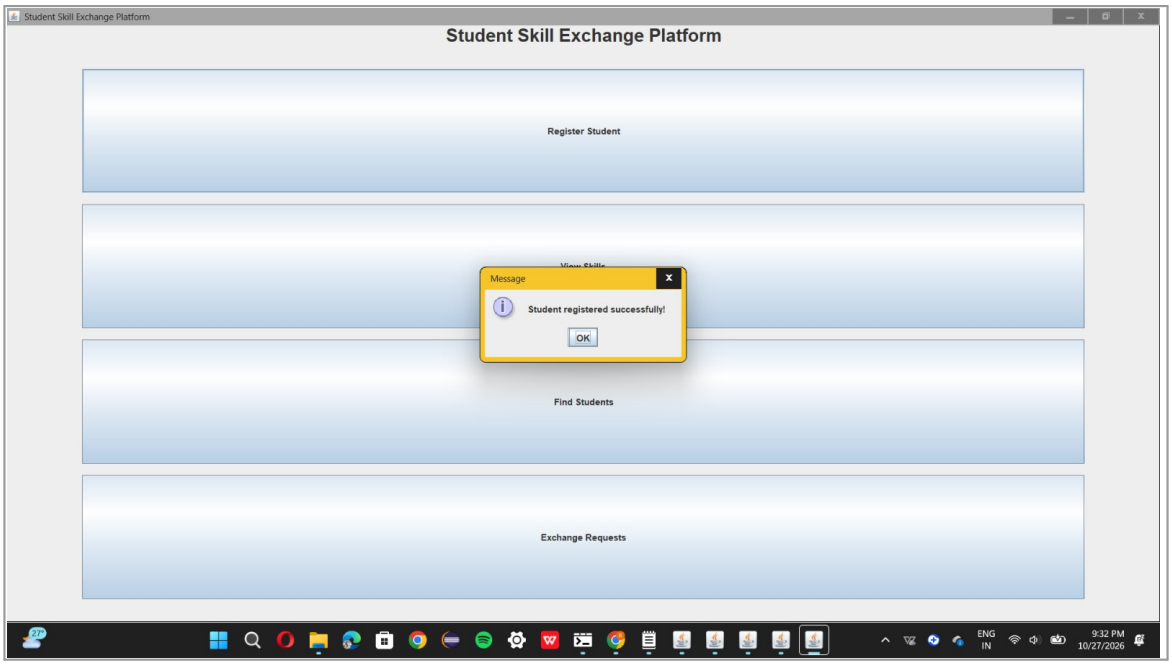


Figure 4: Confirmation message – "Student registered successfully!"

Stage 3: View Skills

The **viewSkills()** method reads data/students.txt line by line, splits each line at the | character and puts the Name, Department, Teach and Learn columns into a `DefaultTableModel`. The model is displayed in a sortable `JTable` inside a dialog box. A message is shown if no records exist.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.io.*;

public class StudentSkillExchange extends JFrame {

    private final String DATA_FILE = "data/students.txt";

    public StudentSkillExchange() {
        setTitle("Student Skill Exchange Platform");
        setSize(700, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JLabel title = new JLabel(
            "Student Skill Exchange Platform",
            SwingConstants.CENTER
        );
        title.setFont(new Font("Arial", Font.BOLD, 24));

        JButton registerButton = new JButton("Register Student");
        JButton skillsButton = new JButton("View Skills");
        JButton searchButton = new JButton("Find Students");
        JButton requestButton = new JButton("Exchange Requests");

        JPanel panel = new JPanel(new GridLayout(4, 1, 15, 15));
        panel.setBorder(
            BorderFactory.createEmptyBorder(30, 100, 30, 100)
        );

        panel.add(registerButton);
        panel.add(skillsButton);
        panel.add(searchButton);
        panel.add(requestButton);

        add(title, BorderLayout.NORTH);
        add(panel, BorderLayout.CENTER);
```

```
        registerButton.addActionListener(e -> registerStudent());
        skillsButton.addActionListener(e -> viewSkills());

        searchButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Student search will be added next.")
        );

        requestButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Exchange requests will be added next.")
        );
    }

    private void registerStudent() {
        JTextField name = new JTextField();
        JTextField email = new JTextField();
        JTextField department = new JTextField();
        JTextField year = new JTextField();
        JTextField teach = new JTextField();
        JTextField learn = new JTextField();

        Object[] fields = {
            "Student Name:", name,
            "Email:", email,
            "Department:", department,
            "Year of Study:", year,
            "Skills to Teach:", teach,
            "Skills to Learn:", learn
        };

        int result = JOptionPane.showConfirmDialog(
            this, fields, "Student Registration",
            JOptionPane.OK_CANCEL_OPTION
        );

        if (result != JOptionPane.OK_OPTION) {
            return;
        }

        if (name.getText().trim().isEmpty()
                || email.getText().trim().isEmpty()
                || department.getText().trim().isEmpty()
                || year.getText().trim().isEmpty()
                || teach.getText().trim().isEmpty()
                || learn.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Please fill in all fields.");
            return;
        }

        try {
            File folder = new File("data");

            if (!folder.exists()) {
                folder.mkdirs();
            }

            try (PrintWriter out = new PrintWriter(
                    new FileWriter(DATA_FILE, true))) {
                out.println(
                    name.getText().trim() + "|"
                    + email.getText().trim() + "|"
                    + department.getText().trim() + "|"
                    + year.getText().trim() + "|"
                    + teach.getText().trim() + "|"
                    + learn.getText().trim()
                );
            }

            JOptionPane.showMessageDialog(this,
                "Student registered successfully!");

        } catch (IOException ex) {
```

```
            JOptionPane.showMessageDialog(this,
                "Could not save student: " + ex.getMessage());
        }
    }

    private void viewSkills() {
        DefaultTableModel model = new DefaultTableModel(
            new String[]{"Name", "Department", "Teach", "Learn"}, 0
        );

        try (BufferedReader br = new BufferedReader(
                new FileReader(DATA_FILE))) {

            String line;

            while ((line = br.readLine()) != null) {
                String[] data = line.split("\\|", -1);

                if (data.length == 6) {
                    model.addRow(new Object[]{
                        data[0], data[2], data[4], data[5]
                    });
                }
            }

        } catch (IOException ex) {
            JOptionPane.showMessageDialog(this,
                "Could not read student data: " + ex.getMessage());
            return;
        }

        if (model.getRowCount() == 0) {
            JOptionPane.showMessageDialog(this,
                "No student records found.");
            return;
        }

        JTable table = new JTable(model);
        table.setAutoCreateRowSorter(true);

        JScrollPane scroll = new JScrollPane(table);
        scroll.setPreferredSize(new Dimension(550, 200));

        JOptionPane.showMessageDialog(this, scroll,
            "Registered Student Skills",
            JOptionPane.PLAIN_MESSAGE);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new StudentSkillExchange().setVisible(true)
        );
    }
}
```

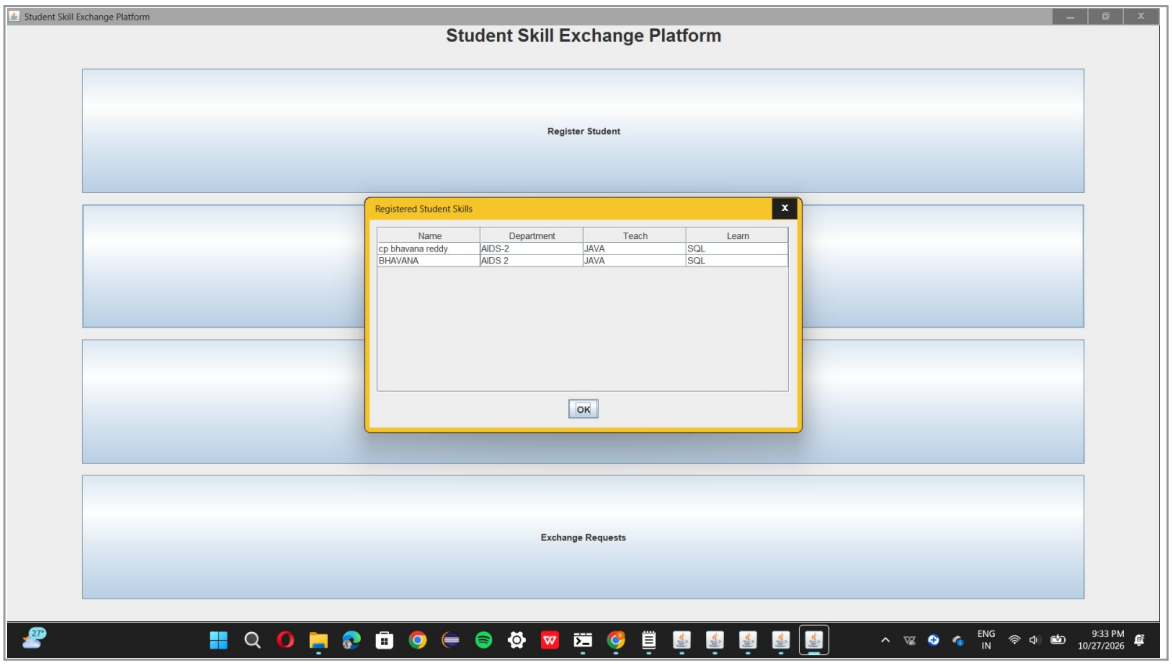


Figure 5: Registered Student Skills table

Stage 4: Find Students

The table-loading code is moved into a reusable method, **loadStudents(query)**. With an empty query it returns every student (used by View Skills); otherwise it returns only the students in whose record any field contains the keyword, ignoring case. **findStudents()** asks for the keyword and shows the matches in a *Search Results* table. Searching for **JAVA** returns both registered students because both of them teach Java.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.io.*;

public class StudentSkillExchange extends JFrame {

    private final String DATA_FILE = "data/students.txt";

    public StudentSkillExchange() {
        setTitle("Student Skill Exchange Platform");
        setSize(700, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JLabel title = new JLabel(
            "Student Skill Exchange Platform",
            SwingConstants.CENTER
        );
        title.setFont(new Font("Arial", Font.BOLD, 24));

        JButton registerButton = new JButton("Register Student");
        JButton skillsButton = new JButton("View Skills");
        JButton searchButton = new JButton("Find Students");
        JButton requestButton = new JButton("Exchange Requests");

        JPanel panel = new JPanel(new GridLayout(4, 1, 15, 15));
        panel.setBorder(
            BorderFactory.createEmptyBorder(30, 100, 30, 100)
        );

        panel.add(registerButton);
        panel.add(skillsButton);
        panel.add(searchButton);
        panel.add(requestButton);
```

```
        add(title, BorderLayout.NORTH);
        add(panel, BorderLayout.CENTER);

        registerButton.addActionListener(e -> registerStudent());
        skillsButton.addActionListener(e -> viewSkills());
        searchButton.addActionListener(e -> findStudents());

        requestButton.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Exchange requests will be added next.")
        );
    }

    private void registerStudent() {
        JTextField name = new JTextField();
        JTextField email = new JTextField();
        JTextField department = new JTextField();
        JTextField year = new JTextField();
        JTextField teach = new JTextField();
        JTextField learn = new JTextField();

        Object[] fields = {
            "Student Name:", name,
            "Email:", email,
            "Department:", department,
            "Year of Study:", year,
            "Skills to Teach:", teach,
            "Skills to Learn:", learn
        };

        int result = JOptionPane.showConfirmDialog(
            this, fields, "Student Registration",
            JOptionPane.OK_CANCEL_OPTION
        );

        if (result != JOptionPane.OK_OPTION) return;

        if (name.getText().trim().isEmpty()
                || email.getText().trim().isEmpty()
                || department.getText().trim().isEmpty()
                || year.getText().trim().isEmpty()
                || teach.getText().trim().isEmpty()
                || learn.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Please fill in all fields.");
            return;
        }

        try {
            File folder = new File("data");
            if (!folder.exists()) folder.mkdirs();

            try (PrintWriter out = new PrintWriter(
                    new FileWriter(DATA_FILE, true))) {
                out.println(
                    name.getText().trim() + "|"
                    + email.getText().trim() + "|"
                    + department.getText().trim() + "|"
                    + year.getText().trim() + "|"
                    + teach.getText().trim() + "|"
                    + learn.getText().trim()
                );
            }

            JOptionPane.showMessageDialog(this,
                "Student registered successfully!");

        } catch (IOException ex) {
            JOptionPane.showMessageDialog(this,
                "Could not save student: " + ex.getMessage());
        }
    }

    private DefaultTableModel loadStudents(String query) {
```

```
        DefaultTableModel model = new DefaultTableModel(
            new String[]{"Name", "Department", "Teach", "Learn"}, 0
        );

        File file = new File(DATA_FILE);
        if (!file.exists()) return model;

        try (BufferedReader br = new BufferedReader(
                new FileReader(file))) {
            String line;

            while ((line = br.readLine()) != null) {
                String[] data = line.split("\\|", -1);

                if (data.length == 6) {
                    boolean matches = query.isEmpty();

                    for (String field : data) {
                        if (field.toLowerCase().contains(query)) {
                            matches = true;
                            break;
                        }
                    }

                    if (matches) {
                        model.addRow(new Object[]{
                            data[0], data[2], data[4], data[5]
                        });
                    }
                }
            }
        } catch (IOException ex) {
            JOptionPane.showMessageDialog(this,
                "Could not read student data: " + ex.getMessage());
        }

        return model;
    }

    private void viewSkills() {
        DefaultTableModel model = loadStudents("");

        if (model.getRowCount() == 0) {
            JOptionPane.showMessageDialog(this,
                "No student records found.");
            return;
        }

        JTable table = new JTable(model);
        table.setAutoCreateRowSorter(true);

        JScrollPane scroll = new JScrollPane(table);
        scroll.setPreferredSize(new Dimension(550, 200));

        JOptionPane.showMessageDialog(this, scroll,
            "Registered Student Skills",
            JOptionPane.PLAIN_MESSAGE);
    }

    private void findStudents() {
        String input = JOptionPane.showInputDialog(this,
            "Enter a name, department, or skill to search:");

        if (input == null) return;

        String query = input.trim().toLowerCase();

        if (query.isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Please enter a search term.");
            return;
        }

        DefaultTableModel model = loadStudents(query);
```

```
            if (model.getRowCount() == 0) {
                JOptionPane.showMessageDialog(this,
                    "No matching students found.");
                return;
            }

            JTable table = new JTable(model);
            JScrollPane scroll = new JScrollPane(table);
            scroll.setPreferredSize(new Dimension(550, 200));

            JOptionPane.showMessageDialog(this, scroll,
                "Search Results", JOptionPane.PLAIN_MESSAGE);
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(() ->
                new StudentSkillExchange().setVisible(true)
            );
        }
}
```

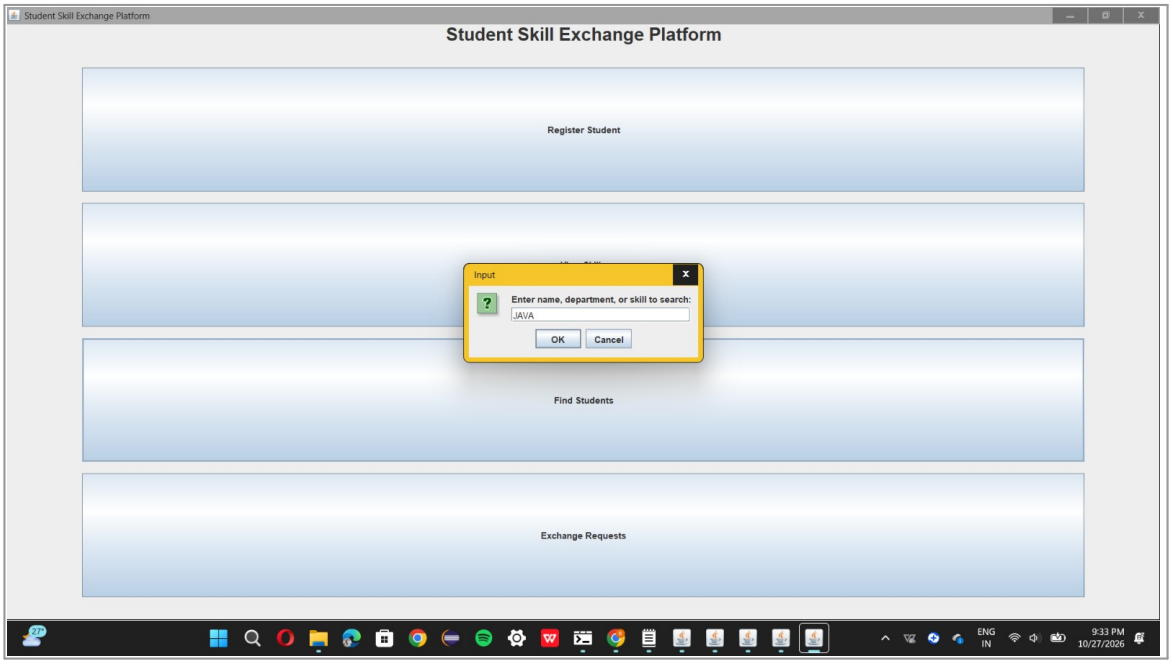


Figure 6: Search input dialog (keyword: JAVA)

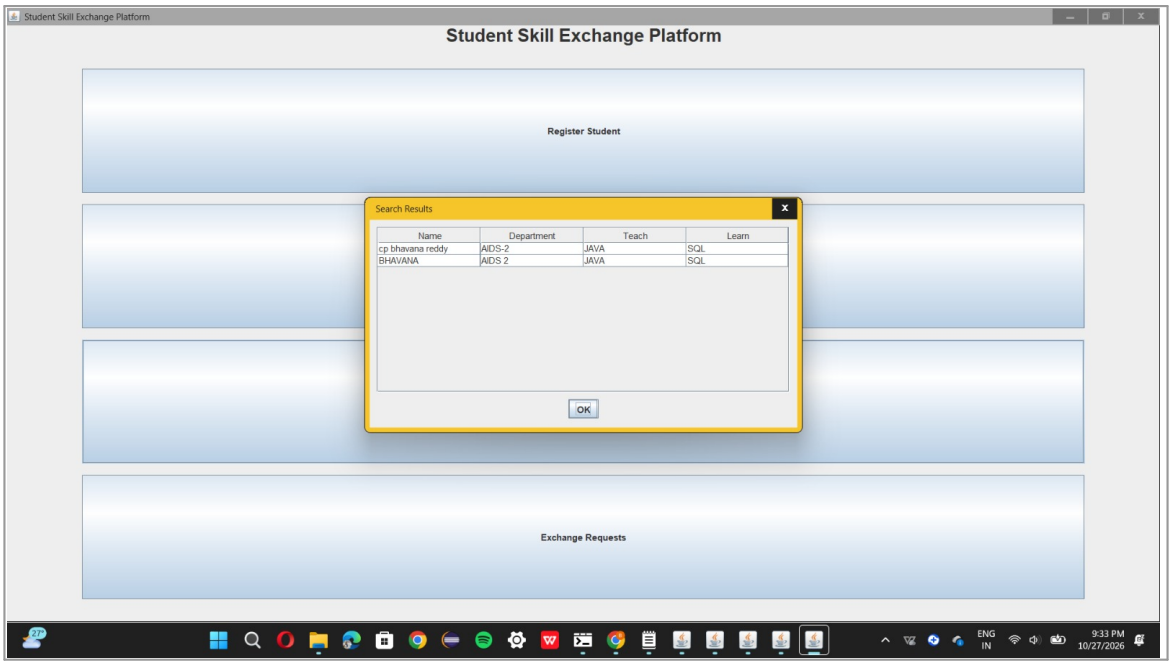


Figure 7: Search Results table

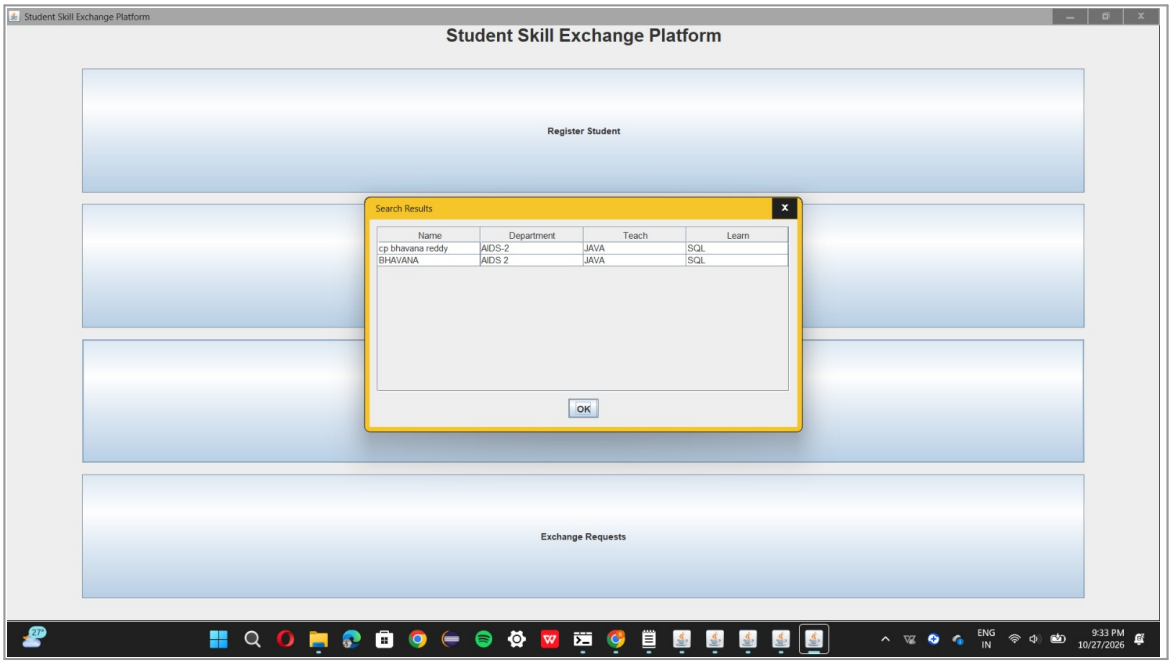


Figure 8: Search Results for the keyword JAVA

Stage 5: Exchange Requests (Final Version)

The final version adds the **Exchange Requests** module and cleans up the program:

- **exchangeRequests()** shows a dialog with *Create Request*, *View Requests* and *Cancel*.
- **createRequest()** saves From, To, Offered and Wanted with the status *Pending* to data/requests.txt; empty fields, the | character, and a request to yourself are rejected.
- **viewRequests()** reads requests.txt into a table with the columns From, To, Offered, Wanted and Status.
- **registerStudent()** now rejects fields that contain | and emails without @, and stores the record with `String.join("|", values)`.

- **showTable()** and **showError()** are shared helper methods used by all modules.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.io.*;

public class StudentSkillExchange extends JFrame {

    private final String DATA_FILE = "data/students.txt";
    private final String REQUEST_FILE = "data/requests.txt";

    public StudentSkillExchange() {
        setTitle("Student Skill Exchange Platform");
        setSize(700, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JLabel title = new JLabel(
            "Student Skill Exchange Platform",
            SwingConstants.CENTER
        );
        title.setFont(new Font("Arial", Font.BOLD, 24));

        JButton registerButton = new JButton("Register Student");
        JButton skillsButton = new JButton("View Skills");
        JButton searchButton = new JButton("Find Students");
        JButton requestButton = new JButton("Exchange Requests");

        JPanel panel = new JPanel(new GridLayout(4, 1, 15, 15));
        panel.setBorder(
            BorderFactory.createEmptyBorder(30, 100, 30, 100)
        );

        panel.add(registerButton);
        panel.add(skillsButton);
        panel.add(searchButton);
        panel.add(requestButton);

        add(title, BorderLayout.NORTH);
        add(panel, BorderLayout.CENTER);

        registerButton.addActionListener(e -> registerStudent());
        skillsButton.addActionListener(e -> viewSkills());
        searchButton.addActionListener(e -> findStudents());
        requestButton.addActionListener(e -> exchangeRequests());
    }

    private void registerStudent() {
        JTextField name = new JTextField();
        JTextField email = new JTextField();
        JTextField department = new JTextField();
        JTextField year = new JTextField();
        JTextField teach = new JTextField();
        JTextField learn = new JTextField();

        Object[] fields = {
            "Student Name:", name,
            "Email:", email,
            "Department:", department,
            "Year of Study:", year,
            "Skills to Teach:", teach,
            "Skills to Learn:", learn
        };

        int result = JOptionPane.showConfirmDialog(
            this, fields, "Student Registration",
            JOptionPane.OK_CANCEL_OPTION
        );

        if (result != JOptionPane.OK_OPTION) return;

        String[] values = {
            name.getText().trim(), email.getText().trim(),
```

```
            department.getText().trim(), year.getText().trim(),
            teach.getText().trim(), learn.getText().trim()
        };

        for (String value : values) {
            if (value.isEmpty() || value.contains("|")) {
                JOptionPane.showMessageDialog(this,
                    "Fill all fields. Do not use | in any field.");
                return;
            }
        }

        if (!values[1].contains("@")) {
            JOptionPane.showMessageDialog(this,
                "Enter a valid email address.");
            return;
        }

        try {
            File folder = new File("data");
            if (!folder.exists()) folder.mkdirs();

            try (PrintWriter out = new PrintWriter(
                    new FileWriter(DATA_FILE, true))) {
                out.println(String.join("|", values));
            }

            JOptionPane.showMessageDialog(this,
                "Student registered successfully!");

        } catch (IOException ex) {
            showError(ex);
        }
    }

    private DefaultTableModel loadStudents(String query) {
        DefaultTableModel model = new DefaultTableModel(
            new String[]{"Name", "Department", "Teach", "Learn"}, 0
        );

        File file = new File(DATA_FILE);
        if (!file.exists()) return model;

        try (BufferedReader br = new BufferedReader(
                new FileReader(file))) {
            String line;

            while ((line = br.readLine()) != null) {
                String[] data = line.split("\\|", -1);

                if (data.length == 6) {
                    boolean matches = query.isEmpty();

                    for (String field : data) {
                        if (field.toLowerCase().contains(query)) {
                            matches = true;
                            break;
                        }
                    }

                    if (matches) {
                        model.addRow(new Object[]{
                            data[0], data[2], data[4], data[5]
                        });
                    }
                }
            }
        } catch (IOException ex) {
            showError(ex);
        }

        return model;
    }
```

```
private void showTable(DefaultTableModel model, String heading) {
    if (model.getRowCount() == 0) {
        JOptionPane.showMessageDialog(this,
            "No records found.");
        return;
    }

    JTable table = new JTable(model);
    table.setAutoCreateRowSorter(true);

    JScrollPane scroll = new JScrollPane(table);
    scroll.setPreferredSize(new Dimension(550, 220));

    JOptionPane.showMessageDialog(this, scroll, heading,
        JOptionPane.PLAIN_MESSAGE);
}

private void viewSkills() {
    showTable(loadStudents(""), "Registered Student Skills");
}

private void findStudents() {
    String input = JOptionPane.showInputDialog(this,
        "Enter name, department, or skill to search:");

    if (input == null) return;

    String query = input.trim().toLowerCase();

    if (query.isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "Enter a search term.");
        return;
    }

    showTable(loadStudents(query), "Search Results");
}

private void exchangeRequests() {
    String[] options = {"Create Request", "View Requests", "Cancel"};

    int choice = JOptionPane.showOptionDialog(
        this, "Choose an action:", "Exchange Requests",
        JOptionPane.DEFAULT_OPTION, JOptionPane.QUESTION_MESSAGE,
        null, options, options[0]
    );

    if (choice == 0) createRequest();
    else if (choice == 1) viewRequests();
}

private void createRequest() {
    JTextField sender = new JTextField();
    JTextField receiver = new JTextField();
    JTextField offered = new JTextField();
    JTextField wanted = new JTextField();

    Object[] fields = {
        "Your Name:", sender,
        "Student Name to Exchange With:", receiver,
        "Skill You Offer:", offered,
        "Skill You Want to Learn:", wanted
    };

    int result = JOptionPane.showConfirmDialog(
        this, fields, "Create Exchange Request",
        JOptionPane.OK_CANCEL_OPTION
    );

    if (result != JOptionPane.OK_OPTION) return;

    String[] values = {
        sender.getText().trim(),
        receiver.getText().trim(),
```

```
            offered.getText().trim(),
            wanted.getText().trim()
        };

        for (String value : values) {
            if (value.isEmpty() || value.contains("|")) {
                JOptionPane.showMessageDialog(this,
                    "Fill all fields. Do not use | in any field.");
                return;
            }
        }

        if (values[0].equalsIgnoreCase(values[1])) {
            JOptionPane.showMessageDialog(this,
                "You cannot request an exchange with yourself.");
            return;
        }

        try {
            File folder = new File("data");
            if (!folder.exists()) folder.mkdirs();

            try (PrintWriter out = new PrintWriter(
                    new FileWriter(REQUEST_FILE, true))) {
                out.println(String.join("|", values) + "|Pending");
            }

            JOptionPane.showMessageDialog(this,
                "Exchange request saved successfully!");

        } catch (IOException ex) {
            showError(ex);
        }
    }

    private void viewRequests() {
        DefaultTableModel model = new DefaultTableModel(
            new String[]{"From", "To", "Offered", "Wanted", "Status"}, 0
        );

        File file = new File(REQUEST_FILE);

        if (!file.exists()) {
            JOptionPane.showMessageDialog(this,
                "No exchange requests found.");
            return;
        }

        try (BufferedReader br = new BufferedReader(
                new FileReader(file))) {
            String line;

            while ((line = br.readLine()) != null) {
                String[] data = line.split("\\|", -1);

                if (data.length == 5) {
                    model.addRow(data);
                }
            }
        } catch (IOException ex) {
            showError(ex);
            return;
        }

        showTable(model, "Saved Exchange Requests");
    }

    private void showError(Exception ex) {
        JOptionPane.showMessageDialog(this,
            "Error: " + ex.getMessage(),
            "File Error", JOptionPane.ERROR_MESSAGE);
    }

    public static void main(String[] args) {
```

```
        SwingUtilities.invokeLater(() ->
            new StudentSkillExchange().setVisible(true)
        );
    }
}
```

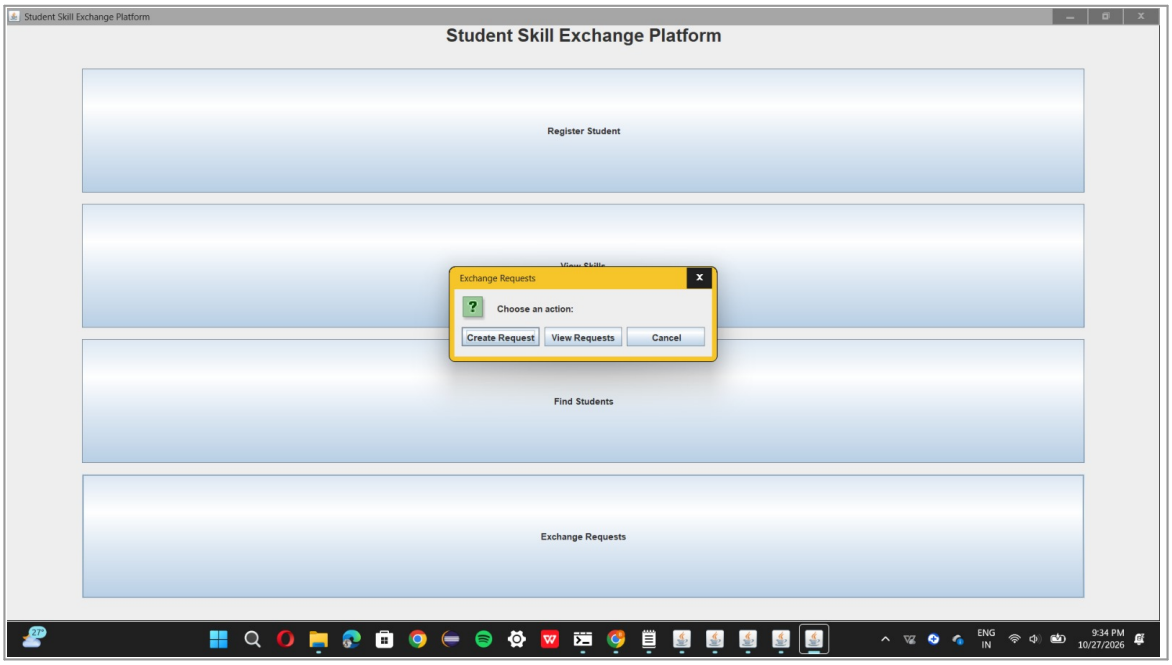


Figure 9: Exchange Requests – choose an action

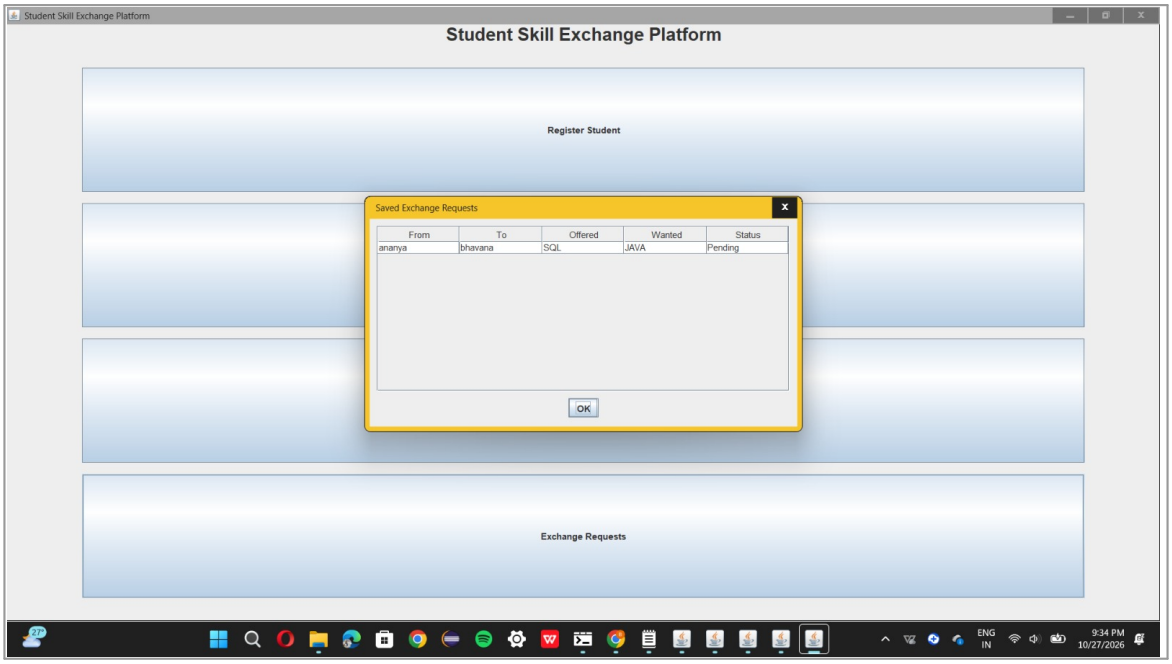


Figure 10: Saved Exchange Requests table (ananya → bhavana, SQL for JAVA, Pending)

7. Database Design (MySQL Schema)

The current program stores its data in text files. For a larger system, the same data can be kept in a MySQL database. The schema below defines two tables: **students** (with a unique email and a hashed password for a future login feature) and **exchange_requests**, which links a sender and a receiver through foreign keys and keeps the request status and creation time.

```
CREATE DATABASE IF NOT EXISTS skill_exchange;
```

```
USE skill_exchange;

CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password_hash CHAR(64) NOT NULL,
    department VARCHAR(100),
    teach_skills TEXT,
    learn_skills TEXT
);

CREATE TABLE exchange_requests (
    id INT PRIMARY KEY AUTO_INCREMENT,
    sender_id INT NOT NULL,
    receiver_id INT NOT NULL,
    skill_name VARCHAR(100) NOT NULL,
    status VARCHAR(20) DEFAULT 'Pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (sender_id) REFERENCES students(id),
    FOREIGN KEY (receiver_id) REFERENCES students(id)
);
```

Table	Purpose	Key columns
students	Registered students and their skills	id (PK), email (unique), department, teach_skills, learn_skills
exchange_requests	Skill-exchange requests between two students	id (PK), sender_id (FK), receiver_id (FK), skill_name, status

8. Test Results

Test	Input	Expected Result	Result
Register student	BHAVANA, AIDS 2, 2026, JAVA, SQL	Record saved; success message	Pass
View skills	Click View Skills	Table of all students	Pass
Find students	Keyword: JAVA	Matching students listed	Pass
Create / view request	ananya → bhavana, SQL / JAVA	Request saved with status Pending	Pass
Validation	Empty field or email without @	Error message shown, nothing saved	Pass
Data persistence	Open students.txt	Records stored one per line	Pass

9. How to Run

```
javac StudentSkillExchange.java
java StudentSkillExchange
```

The program creates a **data** folder next to it, containing students.txt and requests.txt.

10. Conclusion and Future Scope

The Student Skill Management System makes it simple for students to discover peers whose skills match their learning goals. It demonstrates GUI design with Swing, file handling and basic search logic in Java.

- Accept / Reject buttons to update the request status from Pending.
- Move the data to a MySQL database using the schema given in Section 7.
- Login system and email notifications for new requests.
- Stronger validation for the email address and year of study.
- Automatic matching of students whose teach / learn skills complement each other.